2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW) | 979-8-3503-6704-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISSREW63542.2024.00092

State Diagram Extension and Test Case Generation Based on Large Language Models for Improving Test Engineers' Efficiency in Safety Testing

Qingran Su
School of Computer Science and Engineering
Beihang University
Beijing, China
suqingran@buaa.edu.cn

Xingze Li
School of Software
Beihang University
Beijing, China
xingzeli@buaa.edu.cn

Yuming Ren
School of Software
Beihang University
Beijing, China
zy2321206@buaa.edu.cn

Xulang Ouyang
School of Software
Beihang University
Beijing, China
ouyangxulang0703@163.com

Chunming Hu
School of Software
Beihang University
Beijing, China
hucm@buaa.edu.cn

Yongfeng Yin
School of Software
Beihang University
Beijing, China
yyf@buaa.edu.cn

***Abstract*—Safety testing is a key method to ensure software quality. But the quality of the test depends on the level of the test engineers. The method of generating safety test cases based on state diagrams often perform poorly due to lack of personnel experience. Meanwhile, the manual construction of test cases constrains the improvement of safety testing efficiency. This article utilizes LLMs to achieve automated extension and testing of safety state diagrams. The main contributions include extracting safety properties from aviation standards and extending the basic state diagram with LLMs to enable safety testing capabilities. In addition, a genetic algorithm combining edge first depth traversal and LLMs optimization was proposed to improve the efficiency of test case generation. Compared to classical genetic algorithms, the average execution round achieved a 90% reduction. Our LLMs optimization strategy can also enhance other improved genetic algorithms, further improving the efficiency of safety testing.**

***Index Terms*—safety testing, LLMs, safety state diagrams, test case generation**

I. Introduction

Software safety can detect hidden safety vulnerabilities within the system and has received increasing attention, especially in the aerospace industry. The development and certification of avionics systems follow strict standards, such as DO-178C [1] and SAE ARP 4754A [2], ensuring safety and reliability under all conditions. However, these standards lack detailed safety assessments for software and hardware, requiring testers to independently understand how to test safety [3]. The quality of software safety testing heavily relies on the historical experience of testers, often resulting in incomplete safety analysis perspectives. So, engineers with rich experience in safety testing are the most important part of the entire testing work [4].

In recent years, the Model Based Safety Analysis (MBSA) method [5], [6] has been widely applied in the safety analysis and verification of aviation and other safety critical systems [7]. The software state diagram expresses the basic functions of the model. In order to conduct safety analysis systematically, testers need to manually expand the state diagram. The basis for expansion is the common safety properties of the tested software. Due to the varying levels of proficiency among testers, there may be omissions or errors in the state diagram extension, resulting in poor testing quality. And the efficiency of safety testing is difficult to effectively improve. Large Language Models (LLMs) play a revolutionary role in the field of artificial intelligence (AI) [8], [9]. By training on large-scale textual data, LLMs master complex patterns and dependencies in language, demonstrating excellent reasoning and problem-solving abilities [10]. Among these advanced LLMs, models such as ChatGPT, LLaMA, and QWEN [11] have billions of parameters. These models are widely used in content generation and software testing. So we consider using LLMs to automate the state graph for comprehensive extension. This approach can avoid omissions in safety testing caused by inexperienced personnel and improve testing efficiency.

After completing the safety state diagram extension, it is necessary to automatically generate corresponding test cases. Model based test case generation includes two parts: test path generation and input data inference. Test path are subject to varying levels of traversal complexity, and the input data range of a system is typically large [19], making efficient inference of input data challenging. Therefore, finding a suitable set of input data to trigger the test sequence is a challenging task. Based on the widespread application of search algorithms in various fields, the use of genetic algorithms [18] for input data inference is widely applied. But only using genetic algorithms for data inference did not consider the possible data dependencies between transitions [20]. Population size and

2994-810X/24/$31.00 ©2024 IEEE
DOI 10.1109/ISSREW63542.2024.00092

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:19:10 UTC from IEEE Xplore. Restrictions apply.

mutation probability are key parameters in genetic algorithms, and incorrect configuration of these algorithm specific adjustment parameters may lead to convergence to local optima or increase in computational workload. And as the number of problem variables increases, the performance of genetic algorithm may decrease, indicating that it is very sensitive to the dimensionality of the problem [21].

In this paper, we utilize LLMs to automatically extend safety state diagrams and optimize genetic algorithms. By systematically analyzing the mainstream safety standards in the aviation industry, the basis and objectives for conducting safety testing have been obtained. The introduction of LLMs can automatically extend the basic state diagram that expresses functional logic to an extended state diagram that expresses safety properties, reducing reliance on personnel experience. Simultaneously utilizing LLMs to optimize the hyperparameters and individual mutation process of genetic algorithms significantly reduces the execution time of the algorithm. The contributions of this paper are:

Extraction of safety criterion: We extracted specific safety requirements related to state diagram safety extensions from key standards DO-178C and ARP 4754A in the aviation field, and applied them to the extension of safety state diagrams.

Safety state diagram extension based on large language models: We propose a method for safety extension of state diagrams using large language models techniques and have made fine tune to the large language models. By using state diagrams and safety criterion as inputs, and relying on the contextual understanding and generation capabilities of large language models, the state diagrams have been systematically extended.

Test case generation based on state diagram: We propose a test case generation method that integrates edge first depth traversal and genetic algorithm. We use edge first depth traversal algorithm to generate test sequences, and use genetic algorithm to generate corresponding test data. In genetic algorithms, we introduce a large language models to optimize hyper-parameter settings and individual mutation operations.

II. Safety State Diagram Extension and Test Case Generation

This section first introduces the theme framework for extending safety state diagrams and generating test cases. Then, it describes an extension method that combines safety criterion with state diagrams. Finally, it outlines the specific implementation method for generating test cases based on these extended state diagrams.

A. Framework Overview

Figure 1 delineates the systematic process of extracting safety criterion, extending safety state diagrams, and generating test cases, structured into five primary steps. In Step 1, safety criteria are extracted from well-established safety criterion such as DO-178C and APR 4754A. This foundational step ensures that the subsequent processes are rooted in recognized and rigorous safety protocols.

In Step 2, the extracted safety criteria and state diagram XML files are utilized as inputs to a LLMs. Given the unique nature of LLMs, it is crucial to design precise input instructions (prompts) to guide the model effectively. The LLMs processes these inputs, and in Step 3, produces an extended state diagram that incorporates detailed safety status information, thereby enhancing the original state diagram with additional safety insights.

Step 4 involves performing an edge-first depth traversal on the safety-enhanced state diagram generated in Step 3. This traversal method is employed to generate a comprehensive test path specific to the safety state diagram. Finally, in Step 5, a genetic algorithm is used to deduce the input data for the test paths derived in Step 4, resulting in the generation of robust test cases. Notably, during this step, the hyper-parameter settings and mutation algorithms of the genetic algorithm are optimized using the LLMs.

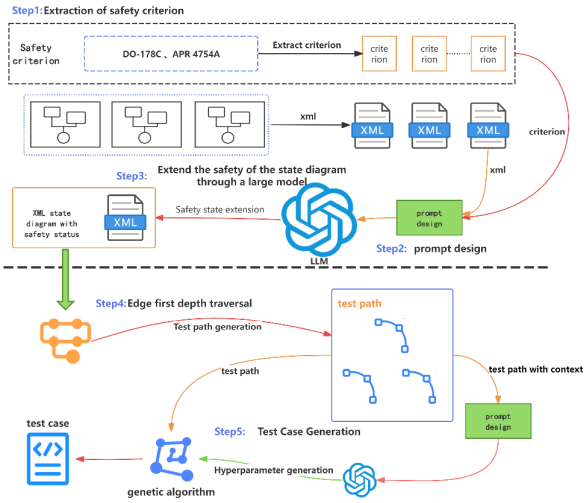

Fig. 1. Overall Architecture

B. Extraction of Safety Criteria

This section further refines and extends these safety criterion to the specific design phase. Figure 2 illustrates the extraction and categorization of safety criteria from the safety criterion DO-178C and APR 4754A into four specific categories. These categories are further broken down into detailed criteria, as shown in the following.

The first category is "The Data Value Exceeds the Range of Values." This category includes criteria such as the output interface data value exceeding the lower limit of the range, the input interface data value exceeding the upper limit of the value range, and whether there is a fault test mechanism for diagnosed fault modes. These criteria ensure that data values remain within safe and expected limits, crucial for maintaining system integrity and reliability.

The second category, "Data Timing Issues." addresses timing-related safety concerns. Specific criteria under this

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:19:10 UTC from IEEE Xplore. Restrictions apply.

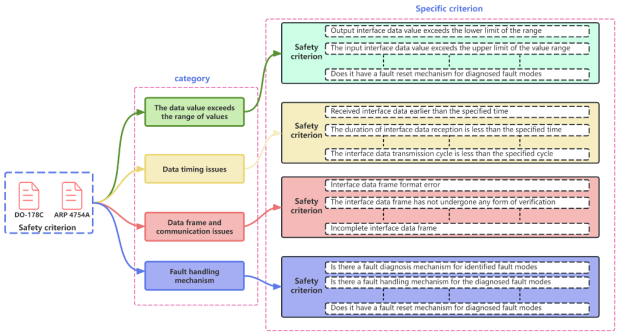

Fig. 2. Extraction and Categorization of Safety Criteria

category include receiving interface data earlier than the specified time, the duration of interface data reception being less than the specified time, and the interface data transmission cycle being shorter than the specified cycle. These criteria are essential for ensuring that the timing of data exchanges adheres to predefined safety criterion, preventing timing discrepancies that could lead to system failures.

The third category is "Data Frame and Communication Issues." This includes criteria such as interface data frame format errors, the interface data frame not undergoing any form of verification, and incomplete interface data frames. Ensuring the integrity and correctness of data frames is vital for reliable communication between system components, preventing miscommunication and data corruption.

The fourth and final category is "Fault Handling Mechanism." This encompasses criteria such as the presence of a fault diagnosis mechanism for identified fault modes, a fault handling mechanism for identified fault modes, and a fault test mechanism for diagnosed fault modes. These criteria ensure that the system can effectively detect, diagnose, and handle faults, maintaining operational safety and minimizing the risk of catastrophic failures. These categories and criteria form the basis for the subsequent processes of extending safety state diagrams and generating test cases, ensuring comprehensive coverage of critical safety aspects defined by the standards.

C. Safety Extension of State Diagram

We first crawled and constructed json format datasets of airborne software and aviation software requirements online, and based on this, fine tuned the model of Qwen2-72B-Instruct using LoRA technology. We carefully selected hyper-parameters, including two different learning rates (5e-05 and 1e-05), training cycle numbers (100 and 50), optimizer (AdamW), learning rate scheduler type (cosine annealing), and LoRA's Dropout rate (0.1), to optimize the model's performance on specific tasks. Through this fine-tuning method, we not only retained the key information and structure of the original model, but also achieved effective representation or approximation through lower dimensional space, significantly improving the performance and generalization ability of the model in the target domain. In the end, we successfully merged and exported the fine tuned LoRA model with the original model, ensuring its consistency and portability, allowing it to be flexibly applied to various practical scenarios. The LoRA fine-tuning process is shown in Figure 3.

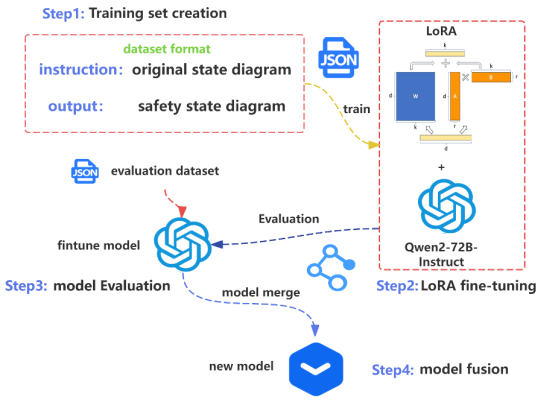

Fig. 3. Extraction and Categorization of Safety Criteria

The selection of prompt words is crucial for the effectiveness of the model output when combining safety criteria and state diagrams as inputs for a large model. In this article, we first provide more detailed and easily understandable prompts to ensure clear input structure, identify key points, and help the model generate accurate output. Secondly, add quality assurance measures after output. This includes model validation and manual review to ensure the accuracy and reliability of the final results. Through these methods, we can effectively reduce the risk of inaccurate information generated by the model. In order to improve the understanding and execution ability of the model, two types of prompt words are designed in this section: one is concise prompt words that do not contain contextual information. Another type is prompt words that contain detailed contextual information.

No context prompt words: This prompt word only takes safety guidelines and state diagrams as input, providing the most basic functional requirement prompts. This approach is suitable for models that already have strong autonomous understanding and reasoning abilities, and can extract key information required to perform tasks from simple instructions.

With context prompt words: Unlike no context prompt words, with context prompt words provide richer information. It first uses specific thinking steps as system prompts and adopts a step-by-step approach to guide the model's specific thinking logic. This method helps the model to understand and process tasks more systematically, ensuring that each step follows the expected logical flow. At the same time, using the format of the state diagram and specific safety criteria as key information inputs provides a clearer execution path and reference basis for the model.

As shown in Figure 4, the design of these two prompt

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:19:10 UTC from IEEE Xplore. Restrictions apply.

words aims to test and optimize the output performance of the model through different information richness. Comparing the two methods, we found that using "context prompt words" generally yields better results. This approach provides structured guidance and clarity, which helps the model process tasks more systematically. As a result, the outputs are more accurate and reliable. To maximize effectiveness, this prompt design requires iterative improvement based on actual system requirements. By continually refining the prompts, we can ensure they align closely with the specific needs of different tasks, further enhancing the model's performance and reducing errors.

D. Test Case Generation

In the testing process based on state diagrams, Depth First Search (DFS) usually consists of two key steps: generating test paths and deducing test data. The edge first depth traversal strategy, compared to node first depth traversal, can more comprehensively cover all transitions in the state graph and better maintain the orderliness between states.

The edge first depth first traversal algorithm recursively traverses all nodes and edges in the graph, recording all independent paths and edges traversed. The algorithm starts from the starting node and gradually delves into each unvisited edge until all edges are accessed. During the traversal process, the current path and visited edges are dynamically updated, and the comprehensiveness and integrity of the traversal are ensured through a backtracking mechanism. This method ensures that all transitions and state sequences in the graph are fully considered and covered.

Genetic algorithm is an optimization algorithm based on natural selection and genetic mechanisms. Its steps include initializing hyper-parameters, generating initial population, calculating individual fitness, individual mutation, individual crossover, selecting individuals, and determining whether individuals meet the requirements. In this section, we introduce LLMs to guide the hyperparameter generation and individual mutation process of genetic algorithms The specific implementation details are as follows:

Hyper-parameter Generation: Using a large model to generate hyper-parameters (POPULATIONIZE, CROSSOVER-RATE, MUTATION-RATE, MAX_GENERATES) for genetic algorithms, by integrating state diagrams and test path information, the large model can make more informed hyper-parameter setting decisions in the context of specific problems. In prompt design, we embed the state diagram and test path information into the prompt prompt words and input them together to the large model. Through this method, the large model can fully understand the specific context of the current problem, thereby generating more accurate and effective hyper-parameter configurations. This not only improves the overall performance of genetic algorithms, but also significantly reduces the workload of manual parameter tuning. Meanwhile, this prompt design that combines state diagrams and test path information ensures that the large model can comprehensively consider the complexity and diversity of test cases when generating hyper-parameters, thereby improving the adaptability and robustness of the algorithm in practical applications.The prompt design is shown in Figure 5.

Population: Each individual in the population consists of N variable values, where N represents the number of variables in the transition trigger conditions in the test path, and, $N \in \mathbb{N}$.

$$\text{Population} = [V_1, V_2, V_3, \ldots, V_N] \tag{1}$$

Each V_i, $i \in \mathbb{N}$, represents the specific value of the i-th variable. During population initialization, the variable values are randomly generated according to their types, which include integer (int) variables, boolean variables, and float variables. Specifically, the population initialization process is as follows:

1. **Integer variables (int):** These variables are randomly generated within a given integer range. If a variable V_i is of type integer, its value is randomly selected between the preset minimum and maximum values.
2. **Boolean variables (boolean):** These variables represent binary values of true or false. During initialization, each boolean variable V_i is equally likely to be set to true or false.
3. **Float variables (float):** These variables are randomly generated within a given floating-point range. If a variable V_i is of type float, its value is randomly selected within the continuous interval between the preset minimum and maximum values.

By randomly generating values for each variable type, the diversity of the initial population can be ensured. This diversity is crucial for the effectiveness of optimization algorithms or evolutionary algorithms, as it provides a rich initial solution space, thereby increasing the likelihood of finding the global optimum.

Fitness Function: The fitness function calculates based on the number of variables triggered in the test path. The test path $path_k$ represents the k-th test path, where $k \in M$ and M represents the total number of test paths.

$$path_k = \{t_1, t_2, \ldots, t_v\} \tag{2}$$

Where $t_j, j \in V$, represents the j-th transition in the test path $path_i$, and V represents the total number of transitions. The calculation formula for the total number of triggered transitions is as (3):

$$\text{ntt} = \sum_{i=1}^{V} \delta(t_j) \tag{3}$$

Where $\delta(t_j)$ is an indicator function. when transition t_j is triggered, $\delta(t_j) = 1$. otherwise, $\delta(t_j) = 0$ and ntt represents number of triggered transitions. Specifically, when transition t_j is triggered, we increment the corresponding variable by 1:

$$\delta(t_j) = \begin{cases} 1 & \text{if } t_j \text{ is triggered} \\ 0 & \text{if } t_j \text{ is not triggered} \end{cases} \tag{4}$$

Based on the above calculations, we can define the fitness function as (5):

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:19:10 UTC from IEEE Xplore. Restrictions apply.

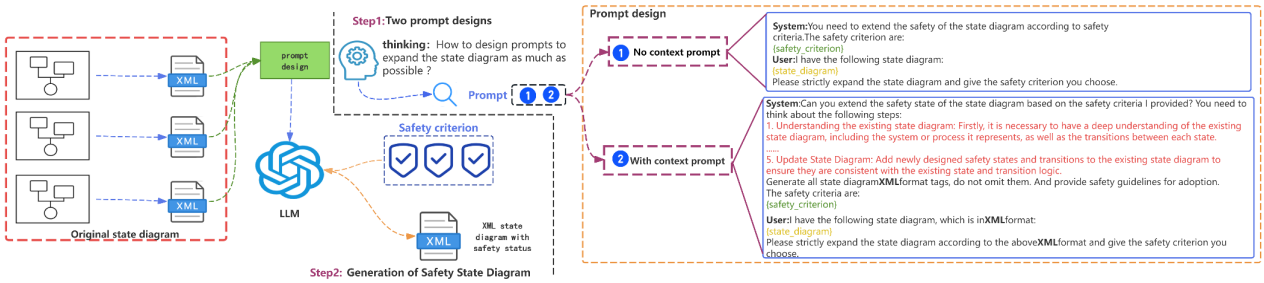

Fig. 4. State diagram extension process and prompt words

System: System: You now need to generate initialization hyperparameters for the genetic algorithm, which include: POPULIONSIZE (representing population size), CROSSOVER-RATE (representing crossover probability), MUTATION-RATE (representing mutation probability), and MAX_GENERATES (representing maximum algebra). You need to refer to the state diagram (in XML format) and the corresponding test path for generation. The state diagram is as follows,Each key value represents the starting node, the first element of each tuple in the value represents the pointed node, and the second element represents the triggered condition:
{state_diagram}
The testing path is as follows:
{test_path}

User: Please provide these hyperparameters so that the genetic algorithm requires fewer iterations to perform better.

Fig. 5. The Prompt of Hyper-parameter Generation

$$\begin{aligned}\text{fitness} &= \alpha \cdot \text{ntt} + \beta \cdot \text{len}(t_1, t_2, \ldots, t_v) \cdot (\alpha + \beta = 1) \\ &= \alpha \sum_{i=1}^{v} \delta(t_j) + \beta \cdot \text{len}(t_1, t_2, \ldots, t_v)\end{aligned} \tag{5}$$

Where len(t_1, t_2, …, t_v) represents the total length of the test path. The weight coefficient α is used to adjust the impact of the number of transitions on the fitness calculation. A larger α value will make the number of transitions contribute more to the fitness. The weight coefficient β is used to adjust the impact of the path length on the fitness calculation. A larger β value will make the path length contribute more to the fitness. In formula (5), the more transitions triggered in the test path and the longer the test path, the greater the fitness value, indicating that the individual is better at solving the problem or more suitable for the target requirements.

Selection: Using sorting selection method to select individuals, first, calculate the fitness value of each individual in the population. The fitness function evaluates an individual's performance in problem-solving based on their characteristics. Sort individuals in the population from high to low based on their fitness values, with individuals with high fitness values ranking first and those with low fitness values ranking last. This step ensures that individuals with high fitness are at the top of the ranking. Each individual is assigned a selection probability based on their position in the ranking. Individuals with higher rankings (i.e. higher fitness values) have a higher probability of selection. Accumulate the selection probabilities assigned to each individual to form a cumulative probability distribution, which can be obtained by gradually adding up the selection probabilities. Generate a random number between 0 and 1, and select an individual based on the cumulative probability distribution.

Individual Mutation: Individual mutations in genetic algorithms have randomness, and during the mutation process, individuals with high fitness may be mutated into individuals with poor fitness, leading to a local optimal state. Here, we use LLMs instead of random mutation of individuals in genetic algorithms. When individuals mutate, it causes each individual to develop towards a higher fitness direction, thereby generating more excellent individuals for screening in genetic algorithms. Due to the consideration of contextual information, in the prompt design process, the state diagram, current test path, transition relationships between paths, and individuals that need to be mutated are used as partial inputs to guide LLMs in individual mutation operations. The specific prompt design is shown in Figure 6.

System: You now need to generate initialization hyperparameters for the genetic algorithm, which include: POPULIONSIZE (representing population size), CROSSOVER-RATE (representing crossover probability), MUTATION-RATE (representing mutation probability), and MAX_GENERATES (representing maximum algebra). You need to refer to the state diagram (in XML format) and the corresponding test path for generation. The state diagram is as follows,Each key value represents the starting node, the first element of each tuple in the value represents the pointed node, and the second element represents the triggered condition:
{state_diagram}
The testing path is as follows:
{test_path}

User: The existing variable is {variable_value},which corresponds to the values of "power", "self Verification", "Error Count", "clipping Attitude", "clipping"; "decision-making", "holdAttitude", and "descent Attitude". Please regenerate these values so that they can pass through all test paths. The generated format should strictly follow the format of {variable_value}and do not generate any other content, Provide only the results, without explanation or other content

Fig. 6. Prompt Design for Individual Mutation

III. Evaluation

In the experimental evaluation, we chose the aviation altitude control system for safety state extension modeling and test case generation. The description of the system is as follows: the altitude control system has two working states: one is to automatically maintain the aircraft flying at the current altitude, abbreviated as the fixed altitude state. Another way is to automatically change the flight altitude until the manually selected altitude is reached, and then maintain the fixed altitude flight, also known as the pre-selected altitude state. When the pilot turns the pre-selected altitude knob to the pre-selected altitude scale, the aircraft automatically enters the climb (or glide) state. After the aircraft approaches the pre-

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:19:10 UTC from IEEE Xplore. Restrictions apply.

selected altitude, it automatically maintains a straight flight at the pre-selected altitude.

Design a state diagram for the system, including initialization Start, InitialVerification, AltitudeMonitor, Climbing, CompleteClimb, Descending, CompleteDescding, Holding. Initialization represents the initial state, and when power is 1, it indicates that the system startup state Start is activated. At this point, verify the system and transition from the Start state to the InitialVerification state. After the system initialization state is fully verified, the Error Count is set to 0 and enters the AltitudeMonitor state. At this time, the aircraft altitude control system simultaneously detects and synchronizes the three variables of climbAltitude, downdAltitude, and holdAltitude. The transition that meets the conditions is triggered and enters the next state. As shown in Figure 7.

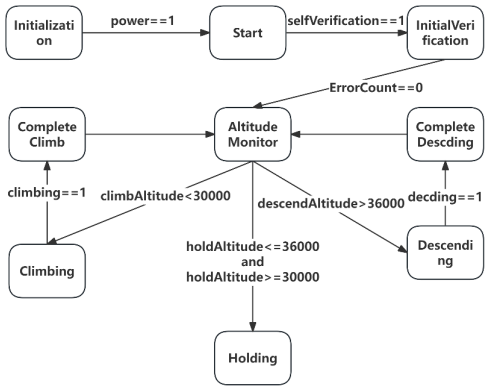

Fig. 7. Original State Diagram

The safety extension of the above state diagram was performed using the fine tuned Qwen2-72B-Instruct model, and the model extended the corresponding safety states based on safety criterion. The detailed information is shown in the Figure 8, and DataValidation is used for data validation in the Safety Extension Module. There are two processing scenarios: 1. When the data is undefined or exceeds the specified range, the system enters the FaultDiagnosis state to diagnose and determine the system's faults. When the fault mode is determined, the system enters the FaultHandling state to repair the system's faults. 2. Check the integrity of the data, and when the data format or structure is incomplete, it will also enter the Fault Diagnosis state and handle the corresponding faults. For ease of representation, the extended safety state diagram is represented in blue as a submodule in the diagram, with arrows of different colors indicating changes after introducing safety states on the basis of the original state diagram. For example, the transition from initialization (represented by yellow dashed line) to power==1 needs to be processed by the safety state module before entering the Start state (represented by yellow dashed line). In terms of verification, we use the UPPAAL tool to validate the safety state diagrams, and combine manual verification to confirm the state diagram. The experiment proves that the extended state graph diagrams to integrity and consistency verification.

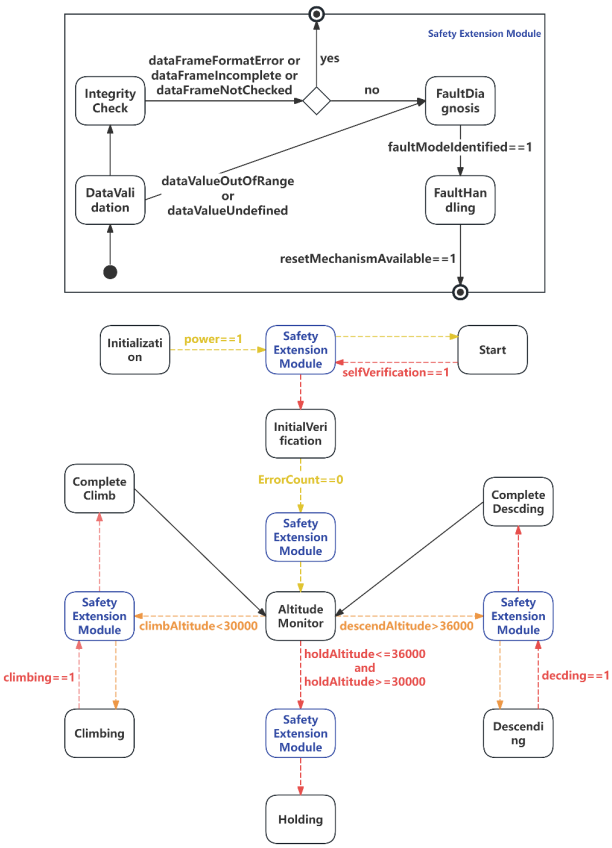

Fig. 8. Safety Extension State Diagram

In genetic algorithms, we compare the genetic algorithm without LLMS intervention (Genetic algorithm), the genetic algorithm using LLMs for hyper-parameter generation (Hyper parameters), and the genetic algorithm using LLMs for individual mutation (Individual mutation). For each testing path, three different genetic algorithms were executed three times and the average was taken. Six different genetic algorithms were used to validate the method, including Path Coverage GA: the genetic algorithm based on test path coverage proposed in the previous section. Self Adaptation GA: Genetic Algorithm with Adaptive Crossover and Mutation Operators [17]. Complex System GA: High efficient complex system genetic algorithm [18]. Negative selection GA: Test data generation of path coverage based on negative selection genetic algorithm [19]. Scalable Parallel GA:A scalable parallel genetic algorithm for the generalized assignment problem [20]. Multiple Paths GA: Multiple paths test case generation based on complex system genetic algorithm [21]. The experimental data is shown in Table I.

From the average number of execution rounds, Individual mutation average (3.60, 12.57, 15.49, 6.78, 2.21, 33.06) is

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:19:10 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EXECUTION ROUNDS COMPARISON OF USING LLMS UNDER DIFFERENT GENETIC ALGORITHMS

Algorithm	round	Path Coverage	Self-Adaption	Complex System	Negative selection	Scalable Parallel	Multiple Paths
Genetic Algorithm	round1	42.6	57.67	60.47	46.52	41.29	78.34
	round2	43.4	58	60.62	44.83	42.33	78.91
	round3	34.2	54.19	51.66	36.68	32.99	75.12
	average	**40.07**	**56.62**	**57.58**	**42.67**	**38.87**	**77.46**
Hyper parameters	round1	23.2	37.53	42.88	26.01	21.36	57.85
	round2	7.2	39.62	51.36	11.00	6.18	59.81
	round3	22.6	28.53	32.81	24.56	20.63	49.02
	average	**17.67**	**35.23**	**42.35**	**20.52**	**16.06**	**55.56**
Individual mutation	round1	3	12.8	9.91	5.68	1.74	33.35
	round2	3.4	11.4	9.67	6.83	2.32	31.51
	round3	4.4	13.5	26.91	7.82	2.57	34.33
	average	**3.6**	**12.57**	**15.49**	**6.78**	**2.21**	**33.06**

better than Hyper parameters average (17.67, 35.23, 42.35, 20.52, 16.06, 55.56) and Genetic average (40.07, 56.62, 57.58, 42.67, 38.87, 77.46). These numbers indicate that genetic algorithms can significantly reduce the number of execution rounds and improve efficiency when using the LLMs for individual mutation. In terms of volatility, Genetic average has the highest fluctuation in the number of execution rounds, with significant differences between different paths. From the data, it can be seen that the performance of these six different genetic algorithms has been improved under the guidance of LLMs. In contrast, the execution round count of individual mutation average is the most stable, and the execution round count of all paths remains at a low level. Based on the above analysis, it can be concluded that the genetic algorithm using the LLMs large model for individual mutation (Individual mutation average) performs the best in terms of execution efficiency and stability. The genetic algorithm using LLMs to generate hyper-parameters (Hyper parameters average) also shows good performance, but may not be as effective as individual mutation methods on certain paths. Traditional genetic algorithms perform poorly in terms of execution rounds and stability, especially on complex paths.

IV. RELATED WORK

This section reviews the most pertinent literature and previous research efforts. In terms of generating test paths based on state diagrams, A UML state diagram based automated statistical test case generation technique has been proposed to improve the efficiency and coverage of object-oriented software testing. By using coverage conversion as the main testing standard and combining it with Rational Software Corporation's Rose RealTime tool, automatic generation of test inputs and expected outputs has been achieved. The application of this method in the case study of avionics systems has verified its effectiveness and feasibility in practical complex system testing [16]. Utilizes the control and data flow logic in the state diagram to automatically traverse the state machine diagram and select conditional predicates on the transformation, then transform these predicates and apply function minimization techniques to generate test cases. This method is capable of generating test cases that meet the transition path coverage criteria, suitable for testing state dependent behaviors at the class and cluster levels, and improving test coverage through boundary testing and predicate coverage. Ultimately, automated test case generation is achieved through a prototype tool called UTG [27]. In the context of genetic algorithms, In related research fields, other researchers have proposed various improvement strategies for genetic algorithms to overcome the limitations of traditional methods. These include the introduction of adaptive crossover and mutation operators, optimization of selection and crossover operators based on complex system theory, fusion of negative selection strategies, implementation of algorithm parallelization, and multi-path coverage test case generation methods for complex systems. These improvements significantly enhance the global search capability, efficiency, and path coverage of the algorithm, bringing significant progress to the field of software testing automation [17]- [21]. LLMs have been effectively used for various coding tasks, including code and unit test generation, presenting unique challenges and opportunities in software testing due to its distinct focus on diverse test inputs and coverage, resulting in innovative practices leveraging LLMs [22]-[25]. In terms of parameter tuning guided by LLMs, LLAMBO enhances Bayesian optimization by integrating LLMs and solving efficient hyper-parameter tuning of Bayesian optimization in data scarcity situations by using LLMs and designing corresponding prompt prompts [26].

V. CONCLUSION AND FUTURE WORK

This article proposes an innovative approach to enhance safety state diagrams and generate test cases. By extracting and analyzing safety properties, more comprehensive testing evidence can be provided for testing engineers, reducing the reliance on personnel experience in the testing process. safety testing quality issues caused by inexperienced testers can be effectively avoided. Meanwhile, LLMs were utilized to extend the safety of state diagrams and automatically generate test cases. The genetic algorithm, which combines edge first depth traversal and large language model optimization, demonstrates significant improvements in test case generation. This method not only improves the coverage of test paths and state nodes, but also generates more independent paths, thereby enhancing the comprehensiveness and reliability of test cases. In addition, by using LLMs to optimize the hyper-parameters

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:19:10 UTC from IEEE Xplore. Restrictions apply.

and individual mutation process of genetic algorithms, the number of execution rounds has been significantly reduced, especially when dealing with complex paths. Therefore, the method proposed in this article is applicable to systems with various similar safety properties and can effectively improve the work efficiency of testing engineers.

We have demonstrated through experiments that our method can automatically extend four types of safety properties. Regarding the testing path coverage of genetic algorithm, the optimized genetic algorithm reduces the maximum number of iterations by about 90%. Regarding various improved genetic algorithms proposed in recent years, the average number of iterations has been reduced by about 77.32%, with the highest reduction being about 94.31%. The results indicate that LLM effectively guides the model optimization process, reduces the model solving process, and improves the efficiency of test case generation.

Based on the identified issues in the current study, the following future work is proposed to address these limitations and enhance the overall approach:

Enrichment of Safety Criteria Extraction: The current safety criteria encompass four main categories and twenty-five sub-items. Future work should focus on expanding these criteria and categories to include additional safety aspects, such as control logic, processing logic, and transfer processes. Achieving this expansion will require ongoing review and integration of new criteria and best practices from a variety of safety domains. By doing so, we can ensure that the criteria remain comprehensive and relevant, effectively addressing emerging safety challenges and technological advancements.

Context-Aware Path Generation: The current method of test path generation does not account for the contextual relationships between different transitions. This oversight can lead to paths with conflicting or incomplete execution conditions. Future work should focus on developing context-aware path generation techniques that consider the dependencies and shared variables between different transitions.

In summary, we utilize the automated extension of state diagrams and the automatic generation of test cases to enhance the safety testing efficiency of testing engineers. Safety testing can be automated and efficiently carried out, reducing reliance on manual operations. The proposed future work aims to address the limitations identified in current research by enriching safety standards, developing robust sample libraries, and incorporating context aware path generation techniques.

ACKNOWLEDGMENT

The authors would like to thank the reviewers for their comments. This work is supported by National Key R&D Program (Grant #2022YFB4501900) and Aviation Science Foundation (Grant #2022Z022051003) of China.

REFERENCES

[1] DO R. 178C,“Software Considerations in Airborne Systems and Equipment Certification,”*RATC Inc.*, Washington, DC, USA, 2011.

[2] Aerospace S. A. E., *ARP4754A: Guidelines for Development of Civil Aircraft and Systems*, SAE International, Warrendale, 2010.

[3] A. Landi and M. Nicholson,“ARP4754A/ED-79A-guidelines for development of civil aircraft and systems-enhancements, novelties and key topics,”*SAE International Journal of Aerospace*, 2011, vol. 4, no. 2011-01-2564, pp. 871–879.

[4] D. Daniels,“Thoughts from the DO-178C committee,”2011.

[5] A. Joshi, M. P. E. Heimdahl, S. P. Miller, et al.,“Model-based safety analysis,”2006.

[6] A. Joshi, S. P. Miller, M. Whalen, et al.,“A proposal for model-based safety analysis,”in *24th Digital Avionics Systems Conference*, IEEE, 2005, vol. 2, pp. 13.

[7] F. Nabi, J. Yong, X. Tao, et al.,“Concepts of safety critical systems unification approach & safety assurance process,”*Journal of Information safety*, vol. 11, no. 4, pp. 292–303, 2020.

[8] A. Radford, K. Narasimhan, T. Salimans, et al.,“Improving language understanding by generative pre-training,”2018.

[9] J. Achiam, S. Adler, S. Agarwal, et al.,“GPT-4 technical report,”*arXiv preprint arXiv:2303.08774*, 2023.

[10] L. Ouyang, J. Wu, X. Jiang, et al.,“Training language models to follow instructions with human feedback,”*Advances in Neural Information Processing Systems*, 2022, vol. 35, pp. 27730–27744.

[11] J. Bai, S. Bai, Y. Chu, et al.,“Qwen technical report,”*arXiv preprint arXiv:2309.16609*, 2023.

[12] K. Derderian, R. M. Hierons, M. Harman, et al.,“Estimating the feasibility of transition paths in extended finite state machines,”*Automated Software Engineering*, 2010, vol. 17, pp. 33–56.

[13] A. Lambora, K. Gupta, K. Chopra,“Genetic algorithm-A literature review,”in *2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COMITCon)*, IEEE, 2019, pp. 380–384.

[14] K. Derderian, R. M. Hierons, M. Harman, et al.,“Generating feasible input sequences for extended finite state machines (EFSMs) using genetic algorithms,”in *Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation*, 2005, pp. 1081–1082.

[15] R. A. Zitar, M. A. Al-Betar, M. A. Awadallah, et al.,“An intensive and comprehensive overview of JAYA algorithm, its versions and applications,”*Archives of Computational Methods in Engineering*, 2022, vol. 29, no. 2, pp. 763–792.

[16] P. Chevalley, P. Thevenod-Fosse,“Automated generation of statistical test cases from UML state diagrams,”in *25th Annual International Computer Software and Applications Conference. COMPSAC 2001*, IEEE, 2001, pp. 205–214.

[17] B. Xiao-An, X. Zi-Jian, Z. Wei,“Approach for Path-oriented Test Cases Generation Based on Improved Genetic Algorithm,”*Computer Science*, 2018, vol. 45, no. 08, pp. 174–178.

[18] J. Zhuang, Q. Y. Yang, H. F. Du,“High efficient complex system genetic algorithm,”*Journal of Software*, 2010, vol. 21, no. 11, pp. 2790–2801.

[19] C. Xia, Y. Zhang, L. Wan, et al.,“Test data generation of path coverage based on negative selection genetic algorithm,”*Acta Electronica Sinica*, 2019, vol. 47, no. 12, pp. 2630–2638.

[20] Y. Y. Liu, S. Wang,“A scalable parallel genetic algorithm for the generalized assignment problem,”*Parallel Computing*, 2015, vol. 46, pp. 98–119.

[21] B. Yu, S. J. Jiang, Y. M. Zhang,“Multiple paths test case generation based on complex system genetic algorithm,”*Computer Science*, 2012, vol. 39, no. 4.

[22] Z. Yuan, Y. Lou, M. Liu, et al.,“No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. CoRR abs/2305.04207 (2023),”*arXiv preprint arXiv:2305.04207*, 2023, vol. 10.

[23] J. Li, G. Li, Y. Li, et al.,“Structured chain-of-thought prompting for code generation,”*arXiv preprint arXiv:2305.06599*, 2023.

[24] J. Li, Y. Zhao, Y. Li, et al.,“Acecoder: Utilizing existing code to enhance code generation,”*arXiv preprint arXiv:2303.17780*, 2023.

[25] Y. Dong, X. Jiang, Z. Jin, et al.,“Self-collaboration code generation via ChatGPT,”*arXiv preprint arXiv:2304.07590*, 2023.

[26] T. Liu, N. Astorga, N. Seedat, et al.,“Large language models to enhance Bayesian optimization,”*arXiv preprint arXiv:2402.03921*, 2024.

[27] P. Samuel, R. Mall, A. K. Bothra,“Automatic test case generation using unified modeling language (UML) state diagrams,”*IET Software*, 2008, vol. 2, no. 2, pp. 79–93.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:19:10 UTC from IEEE Xplore. Restrictions apply.